DSA Lecture Notes: ADT and Arrays

Simple English | Java examples | Based on your ~13-minute Hindi YouTube lecture transcript

Big idea: An Abstract Data Type (ADT) tells us WHAT operations are possible. The implementation tells us HOW they work.

1. What is a data type? (around 1:45-3:15)

A data type tells Java what kind of value a variable can hold and which operations make sense. Think of a variable as a labelled box.

```
int x = 10;
int y = 5;
System.out.println(x + y); // 15
```

Here, int is the type, x is the variable name, 10 is its value, and + is an operation.

2. Primitive and reference types (around 3:15-4:00)

Primitive types are built into Java: byte, short, int, long, float, double, char, boolean. Reference types include String, arrays, and class objects. You can also define your own classes.

```
double price = 99.5;
String name = "Java";
int[] marks = {80, 90, 75};
```

3. What is abstraction? (around 4:00-7:45)

Abstraction means showing useful actions while hiding the internal details. Example: you press a lift button to go up; you do not need to know how the motor is controlled.

Remember: WHAT it does = visible behaviour. HOW it does it = hidden implementation.

4. What is an Abstract Data Type? (around 7:50-9:35)

An ADT is a description of a collection of data and the operations it supports, without choosing a particular internal implementation.

For example, a Stack ADT says you can push, pop, peek, and check whether it is empty. It does not say whether the stack must use an array or linked list.

5. Arrays in the lecture (around 9:35-11:15)

The instructor uses an array-like collection to explain operations such as add, remove, and get. These are examples of behaviour, not necessarily Java array methods.

```
int[] nums = {10, 20, 30};
System.out.println(nums[1]); // 20
nums[1] = 25;               // update element
```

Important Java detail: A Java array has a fixed length. It has no built-in push() or remove() methods. To add/remove items dynamically, a class such as ArrayList is often used.

```
import java.util.ArrayList;
ArrayList<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
list.remove(0);
```

```
System.out.println(list.get(0)); // 20
```

6. Stack ADT and LIFO (around 11:20-13:00)

Imagine stacking books. The last book you put on top is the first book you take off. This is LIFO: Last In, First Out.

Operation	Easy meaning
push(x)	Put x on top
pop()	Remove and return the top item
peek()	Look at the top without removing it
isEmpty()	Check whether the stack has no items

```
import java.util.ArrayDeque;
import java.util.Deque;

Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.push(20);
stack.push(30);
System.out.println(stack.peek()); // 30
System.out.println(stack.pop());  // 30
System.out.println(stack.peek()); // 20
```

7. ADT vs data structure: interview answer

ADT: Defines the operations and rules (WHAT). Data structure/implementation: Decides how values are stored and how those operations are performed (HOW). One ADT can have different implementations.

Example: A stack can be implemented with an array or a linked list. Both follow LIFO.

8. Quick revision and practice

1. What does a data type tell us? 2. What does abstraction hide? 3. What does ADT define? 4. What is LIFO? 5. Difference between peek() and pop()? 6. Can a Java array grow after creation?

Answers: 1. Value type and valid operations. 2. Internal implementation. 3. Behaviour/operations. 4. Last In, First Out. 5. peek reads top; pop removes top. 6. No, array length is fixed.

One-minute summary

Data type = kind of value. Abstraction = hide HOW. ADT = define WHAT operations are supported. Array = fixed-length indexed collection in Java. Stack = LIFO collection with push/pop/peek/isEmpty.

Source scope: This lecture mainly teaches the ADT concept using arrays and stacks. The Java code examples and Java-array clarification are added to make the ideas easier to practise.